

```
In [1]: # Experiment 2 - Step 1
# Import required libraries
import cv2
import numpy as np
import matplotlib.pyplot as plt

print("OpenCV version:", cv2.__version__)
print("NumPy version:", np.__version__)
print("Libraries imported successfully!")
```

OpenCV version: 5.0.0
NumPy version: 2.5.2
Libraries imported successfully!

```
In [7]: # Step 2: Load image and convert to grayscale

image = cv2.imread("exp2.jpeg")

if image is None:
    print("Image not found!")
else:
    print("Image loaded successfully!")

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

print("Original image shape:", image.shape)
print("Grayscale image shape:", gray.shape)
```

Image loaded successfully!
Original image shape: (360, 360, 3)
Grayscale image shape: (360, 360)

```
In [8]: # Step 3: Display original grayscale image

plt.figure(figsize=(6, 6))

plt.imshow(gray, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")

plt.show()
```

Original Grayscale Image



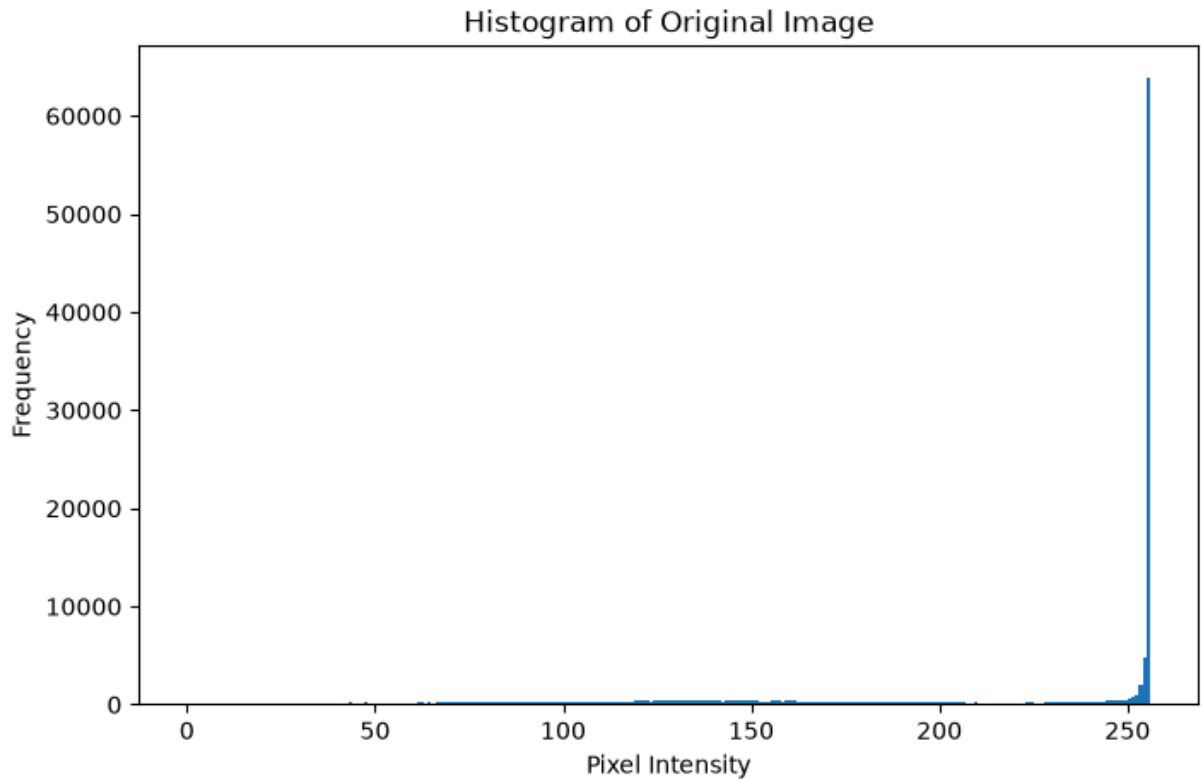
```
In [9]: # Step 4: Generate histogram of original image

plt.figure(figsize=(8, 5))

plt.hist(gray.ravel(), bins=256, range=[0, 256])

plt.title("Histogram of Original Image")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

plt.show()
```



In [10]: *# Step 5: Contrast Stretching*

```
min_val = np.min(gray)
max_val = np.max(gray)

contrast_stretched = ((gray - min_val) / (max_val - min_val) * 255).astype(n

print("Minimum intensity before:", min_val)
print("Maximum intensity before:", max_val)

print("Minimum intensity after:", np.min(contrast_stretched))
print("Maximum intensity after:", np.max(contrast_stretched))
```

Minimum intensity before: 0
Maximum intensity before: 255
Minimum intensity after: 0
Maximum intensity after: 255

In [11]: *# Display contrast stretched image*

```
plt.figure(figsize=(6, 6))

plt.imshow(contrast_stretched, cmap="gray")
plt.title("Contrast Stretched Image")
plt.axis("off")

plt.show()
```

Contrast Stretched Image



```
In [12]: # Step 6: Histogram Equalization

equalized = cv2.equalizeHist(gray)

plt.figure(figsize=(6, 6))

plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalized Image")
plt.axis("off")

plt.show()
```

Histogram Equalized Image



```
In [14]: # Step 7: CLAHE

clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

clahe_image = clahe.apply(gray)

plt.figure(figsize=(6, 6))

plt.imshow(clahe_image, cmap="gray")
plt.title("CLAHE Enhanced Image")
plt.axis("off")

plt.show()
```

CLAHE Enhanced Image



```
In [15]: # Step 8: Compare Histograms

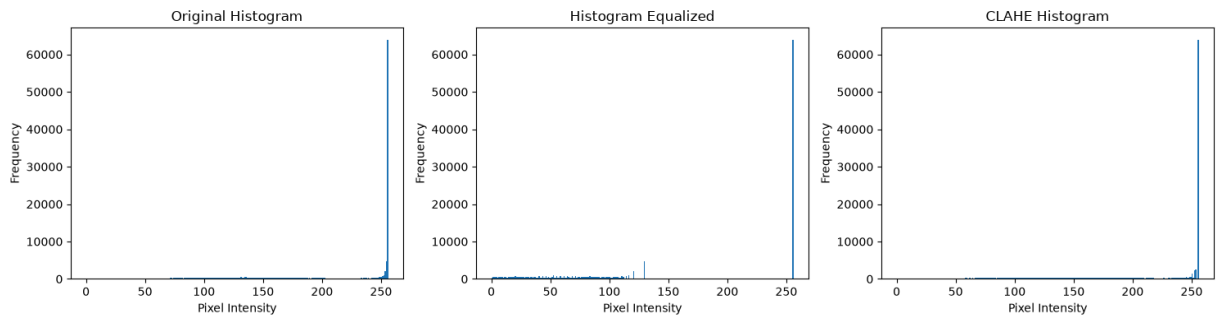
plt.figure(figsize=(15, 4))

# Original histogram
plt.subplot(1, 3, 1)
plt.hist(gray.ravel(), bins=256, range=[0, 256])
plt.title("Original Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

# Equalized histogram
plt.subplot(1, 3, 2)
plt.hist(equalized.ravel(), bins=256, range=[0, 256])
plt.title("Histogram Equalized")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

# CLAHE histogram
plt.subplot(1, 3, 3)
plt.hist(clahe_image.ravel(), bins=256, range=[0, 256])
plt.title("CLAHE Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
```

```
plt.tight_layout()
plt.show()
```



In [16]: *# Step 9: Final Visual Comparison*

```
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(contrast_stretched, cmap="gray")
plt.title("Contrast Stretching")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalization")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(clahe_image, cmap="gray")
plt.title("CLAHE")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Original



Contrast Stretching



Histogram Equalization



CLAHE



Observation

The original grayscale image was analyzed using its intensity histogram. Contrast stretching improved the dynamic range of pixel intensities. Histogram Equalization enhanced the overall contrast by redistributing pixel intensities. CLAHE provided local contrast enhancement and was useful for improving details in regions with varying illumination.

Overall, histogram-based enhancement techniques improved the visibility of image features and can be useful as preprocessing techniques for computer vision applications.

Conclusion

In this experiment, histogram-based image enhancement techniques were successfully implemented using Python and OpenCV. Contrast stretching, Histogram

Equalization, and CLAHE were applied to a grayscale image. The original and enhanced images and their histograms were compared.

The experiment demonstrated that histogram-based preprocessing can improve contrast, brightness distribution, and feature visibility. CLAHE is particularly useful for images with non-uniform illumination and is widely used in medical imaging and low-light image enhancement.